

MASTER PYTHON PRACTICAL ASSIGNMENT

100 Problem Statements — From Python Basics to Advanced Core Python

Technology: Python 3.x **External Libraries:** Not Required

Complete these practical problems progressively. The assignment is designed to build Python skills from fundamentals through advanced core Python concepts. No evaluation criteria or marks are included.

Python Fundamentals

Task 1: Hello Python

Write a program that displays your name, age, city, and a short introduction. Store each value in a variable.

Task 2: Personal Information

Create variables for a person's name, age, email, mobile number, city, and profession. Display a formatted profile.

Task 3: Simple Calculator

Accept two numbers and display addition, subtraction, multiplication, division, floor division, modulus, and power.

Task 4: Area and Perimeter

Accept rectangle dimensions and calculate area and perimeter.

Task 5: Circle Calculator

Accept radius and calculate area and circumference using a constant for pi.

Task 6: Temperature Converter

Convert Celsius to Fahrenheit and Kelvin.

Task 7: Time Converter

Convert a number of seconds into hours, minutes, and remaining seconds.

Task 8: Currency Breakdown

Accept an amount and determine the number of notes required for selected denominations.

Task 9: Simple Interest

Accept principal, rate, and time and calculate simple interest and total amount.

Task 10: Salary Calculator

Accept basic salary, allowance percentage, and deduction percentage and calculate net salary.

Conditions

Task 11: Positive, Negative or Zero

Accept a number and determine whether it is positive, negative, or zero.

Task 12: Even or Odd

Determine whether an integer is even or odd.

Task 13: Largest of Two

Find the larger of two numbers without using max().

Task 14: Largest of Three

Find the largest of three numbers using conditional statements.

Task 15: Leap Year

Determine whether a year is a leap year.

Task 16: Voting Eligibility

Check voting eligibility based on age and handle invalid ages.

Task 17: Grade Calculator

Accept marks and calculate a grade. Reject marks outside the valid range.

Task 18: Electricity Bill

Calculate an electricity bill using slab-based rates.

Task 19: Login Validation

Create a simple username/password login with a limited number of attempts.

Task 20: Triangle Validator

Validate three sides and classify a valid triangle as equilateral, isosceles, or scalene.

Loops

Task 21: Print 1 to N

Accept N and print numbers from 1 through N.

Task 22: Reverse Counting

Print numbers from N down to 1.

Task 23: Multiplication Table

Print the multiplication table of a number up to 20.

Task 24: Sum of Natural Numbers

Calculate the sum from 1 to N using a loop.

Task 25: Factorial

Calculate factorial without using `math.factorial()`.

Task 26: Count Digits

Count the digits in an integer.

Task 27: Reverse Number

Reverse an integer using arithmetic and loops.

Task 28: Palindrome Number

Determine whether an integer is a palindrome without converting it to a string.

Task 29: Prime Number

Determine whether a number is prime.

Task 30: Prime Numbers in Range

Display all prime numbers between two limits.

Task 31: Fibonacci Series

Generate the first N Fibonacci terms iteratively.

Task 32: GCD and LCM

Calculate GCD and LCM of two positive integers.

Task 33: Armstrong Number

Determine whether a number is an Armstrong number.

Task 34: Perfect Number

Determine whether a number is a perfect number.

Task 35: Number Guessing Game

Create a guessing game that reports the number of attempts.

Strings

Task 36: String Statistics

Count uppercase letters, lowercase letters, digits, spaces, and special characters.

Task 37: Reverse a String

Reverse a string without using slicing.

Task 38: Palindrome String

Check whether a string is a palindrome while ignoring case.

Task 39: Character Frequency

Count occurrences of every character in a string.

Task 40: Word Frequency

Count the frequency of every word in a sentence.

Task 41: Longest Word

Find the longest word without using `max()` with a key function.

Task 42: Remove Duplicate Characters

Remove repeated characters while preserving first occurrence order.

Task 43: Anagram Checker

Determine whether two strings are anagrams, ignoring spaces and case.

Task 44: Vowel and Consonant Counter

Count vowels and consonants while ignoring non-letters.

Task 45: Password Strength Checker

Validate minimum length, uppercase, lowercase, digit, and special-character requirements.

Lists

Task 46: Student Marks List

Store subject marks in a list and calculate total, average, highest, and lowest.

Task 47: Remove Duplicates

Remove duplicate list values while preserving order.

Task 48: Second Largest

Find the second-largest unique number without sorting.

Task 49: List Rotation

Rotate a list left or right by a requested number of positions.

Task 50: Merge Lists

Merge two lists into a unique third list.

Task 51: Common Elements

Find common elements between two lists without set intersection.

Task 52: Separate Even and Odd

Create separate even and odd lists.

Task 53: Positive and Negative Lists

Separate positive, negative, and zero values.

Task 54: List Statistics

Calculate count, sum, average, minimum, maximum, and range.

Task 55: Shopping Cart

Create a cart supporting add, remove, quantity update, and total calculation.

Tuples, Sets and Dictionaries

Task 56: Employee Tuple

Store employee ID, name, department, and salary in a tuple and use unpacking.

Task 57: Tuple Search

Store student tuples and find students above a specified marks threshold.

Task 58: Set Operations

Given two groups, calculate union, intersection, difference, and symmetric difference.

Task 59: Unique Words

Count unique words in a paragraph after normalizing case.

Task 60: Dictionary Profile

Create a nested employee dictionary and display selected fields.

Task 61: Student Result Dictionary

Store students and subject marks in dictionaries and calculate totals and averages.

Task 62: Word Frequency Dictionary

Build a dictionary containing word frequencies for a paragraph.

Task 63: Inventory Dictionary

Manage products, quantity, price, stock updates, and total inventory value.

Task 64: Phone Book

Build a dictionary-based phone book with add, search, update, delete, and display.

Task 65: Dictionary Sorting

Sort student records by marks in descending order without changing the original dictionary.

Functions

Task 66: Reusable Calculator

Create separate functions for arithmetic operations.

Task 67: Temperature Functions

Create reusable temperature conversion functions.

Task 68: Prime Function

Write `is_prime()` returning True or False and reuse it in a range program.

Task 69: Factorial Function

Create `factorial()` and use it for combinations and permutations.

Task 70: Statistics Functions

Create reusable functions for total, average, minimum, maximum, and median.

Task 71: Student Result Function

Create a function returning total, average, grade, and pass/fail.

Task 72: Default and Keyword Arguments

Create an employee-profile function using default and keyword arguments.

Task 73: Variable-Length Arguments

Create a function accepting any number of numbers and returning summary statistics.

Task 74: Higher-Order Function

Create a function that accepts another function and applies it to list items.

Task 75: Menu-Driven Utility

Build a menu-driven console application using separate functions for operations.

Recursion

Task 76: Recursive Factorial

Calculate factorial recursively.

Task 77: Recursive Fibonacci

Generate Fibonacci terms recursively and observe its performance limitation.

Task 78: Recursive Sum

Calculate the sum from 1 to N recursively.

Task 79: Recursive Digit Sum

Calculate the sum of an integer's digits recursively.

Task 80: Recursive String Reverse

Reverse a string using recursion.

Task 81: Recursive Palindrome

Check a string for palindrome using recursion.

Task 82: Recursive GCD

Implement Euclid's GCD algorithm recursively.

Task 83: Tower of Hanoi

Implement Tower of Hanoi recursively and display each move.

Task 84: Recursive Hierarchy

Represent a hierarchy with nested dictionaries and recursively print it.

Task 85: Recursive Nested Sum

Calculate the sum of numbers in a nested list recursively.

Files and Data Persistence

Task 86: Create a Text File

Save user notes to a text file and allow additional notes to be appended.

Task 87: Read and Display File

Read a text file and display its contents with line numbers.

Task 88: File Statistics

Calculate lines, words, characters, and non-space characters in a file.

Task 89: Search in File

Search for a keyword and display matching lines with line numbers.

Task 90: Copy File

Copy one text file to another using file handling.

Task 91: Student File Records

Store student records in a text file and support add, search, update, and display.

Task 92: CSV-Style File

Create and read a comma-separated file without importing the csv library.

Task 93: JSON Data Storage

Save and load employee records using Python's built-in json module.

Task 94: Configuration File

Read and update selected settings in a JSON configuration file.

Task 95: File Backup Utility

Copy selected data files into a backup folder using built-in file/path functionality.

Exception Handling

Task 96: Safe Division

Build a calculator that handles division by zero and invalid numeric input.

Task 97: Validated Integer Input

Create a function that repeatedly requests an integer until valid.

Task 98: Custom Error Message

Build a marks-entry program that handles invalid marks.

Task 99: Bank Withdrawal Errors

Handle insufficient balance, invalid amount, and invalid input.

Task 100: File Error Handling

Handle missing files, permission errors, and invalid content.

Task 101: Custom Exception

Create an InvalidAgeError and use it in an eligibility program.

Task 102: Password Exceptions

Create custom exceptions for weak or invalid passwords.

Task 103: Transaction Processor

Process transactions so one invalid transaction does not stop the entire batch.

Task 104: Safe JSON Loader

Load JSON safely and handle missing files and invalid JSON.

Task 105: Robust Menu Application

Create a menu program that handles unexpected input without crashing.

Object-Oriented Programming

Task 106: Student Class

Create Student with ID, name, marks, and methods for total, average, and grade.

Task 107: Employee Class

Create Employee with details and annual-salary calculation.

Task 108: Bank Account Class

Create BankAccount with deposit, withdrawal, balance, and transaction history.

Task 109: Constructor Practice

Create Product demonstrating constructor parameters and defaults.

Task 110: Encapsulation

Create a class with controlled access to a balance or sensitive attribute.

Task 111: Inheritance

Create Person as a base class and Student and Trainer as derived classes.

Task 112: Method Overriding

Create employee subclasses that override calculate_salary().

Task 113: Class Variable

Maintain a class-level count of created Employee objects.

Task 114: Static and Class Methods

Demonstrate meaningful staticmethod and classmethod use.

Task 115: Property Decorators

Create a Product price property that validates values before updating.

Advanced Core Python

Task 116: Magic Methods

Create Book implementing __str__() and __repr__().

Task 117: Object Comparison

Allow Student objects to be compared by marks.

Task 118: Operator Overloading

Create Vector supporting addition through a special method.

Task 119: Iterator Class

Create a custom iterator generating values from start to end.

Task 120: Generator Function

Create a generator that produces Fibonacci numbers one at a time.

Task 121: Large-File Generator

Create a generator that reads a large text file line by line.

Task 122: Logging Decorator

Create a decorator recording function start, end, and execution duration.

Task 123: Validation Decorator

Create a decorator that validates function arguments before execution.

Task 124: Closure Counter

Create a function returning another function that maintains a private counter.

Task 125: Lambda and Sorting

Sort employee records using lambda expressions by salary, department, and name.

Advanced Core Python Applications

Task 126: Contact Management System

Build a console contact manager using functions, dictionaries, validation, exceptions, and file persistence.

Task 127: Library Management System

Build a library system for books, members, issue/return operations, due dates, and persistent storage.

Task 128: Banking System

Build banking software using classes, inheritance, custom exceptions, transaction history, and persistence.

Task 129: Inventory Management System

Manage products, stock-in, stock-out, search, low-stock detection, and persistent data.

Task 130: Employee Payroll System

Build payroll using classes, inheritance, salary calculations, deductions, allowances, and persistence.

Task 131: Student Management System

Build student registration, marks, attendance, grades, search, update, delete, and reports.

Task 132: Expense Tracker

Build an expense tracker with categories, dates, amounts, summaries, filtering, and JSON persistence.

Task 133: Quiz Application

Load questions from JSON, accept answers, calculate scores, and generate a final report.

Task 134: Task Management System

Build task creation, priorities, due dates, status changes, searching, filtering, and persistent storage.

Task 135: Mini Banking and Reporting System

Combine OOP, custom exceptions, decorators, generators/iterators, JSON persistence, transactions, reporting, and a modular menu-driven interface.

General Instructions

- Use Python 3.x.
- Do not use external libraries or frameworks. Focus on Python language features and built-in/standard functionality.
- Write clean, readable, reusable code with meaningful names.
- Validate user input wherever appropriate.
- Handle expected errors gracefully.
- For file tasks, create and manage the required files through your program.
- For advanced tasks, use modular functions and classes where appropriate.
- Do not simply copy solutions. Be prepared to explain your logic.
- For the final application tasks, combine concepts learned in earlier tasks.